

Sistema de Gestão de Loja de Videojogos

Tomás Sousa 1251515 Bruno Sousa 1230748

Fundamentos de Desenvolvimento de Software

Resumo

O presente relatório descreve a terceira iteração do projeto Sistema de Gestão de Loja de Videojogos, desenvolvido no âmbito da unidade curricular Fundamentos de Desenvolvimento de Software. O sistema foi implementado em C++ e tem como objetivo apoiar a gestão das operações de uma loja de videojogos, permitindo a administração de clientes, empregados, produtos e vendas. Nesta iteração foi consolidada a arquitetura da aplicação, reforçada a separação de responsabilidades através de uma abordagem inspirada no padrão Model-View-Controller (MVC) e introduzidos mecanismos de tratamento de exceções e processamento de vendas.

Palavras-chave: C++, MVC, Programação Orientada a Objetos, Gestão de Vendas, Engenharia de Software.

1. Introdução

O desenvolvimento de software moderno exige a aplicação de princípios de modularidade, reutilização e manutenção. Neste contexto, foi desenvolvido um sistema de gestão de uma loja de videojogos capaz de suportar operações relacionadas com clientes, empregados, produtos e vendas.

Esta terceira iteração representa uma evolução significativa relativamente às versões anteriores, incorporando novas funcionalidades relacionadas com o processamento de vendas, controlo de stock e tratamento estruturado de exceções.

2. Modelo de Domínio

As principais entidades do sistema são:

- Cliente
- Empregado

- Produto
- Venda
- ItemVenda
- Pagamento

2.1 Cliente

Representa um cliente registado na loja, contendo informação como nome, NIF e email.

2.2 Empregado

Representa um colaborador responsável pela execução das operações da loja.

2.3 Produto

Representa um videojogo ou outro artigo comercializado pela loja.

2.4 Venda

Representa uma transação comercial efetuada por um cliente.

2.5 ItemVenda

Representa uma linha individual de uma venda, associando um produto à respetiva quantidade.

2.6 Pagamento

Representa a informação associada ao pagamento efetuado pelo cliente.

3. Design

3.1 Arquitetura Física

O projeto encontra-se dividido nos seguintes módulos:

- controller
- model
- views
- exceptions
- mock

3.2 Arquitetura Lógica

A aplicação segue uma arquitetura inspirada no padrão MVC.

Model – Responsável pela representação dos dados e regras de negócio.

View – Responsável pela interação com o utilizador através da consola.

Controller – Responsável pela coordenação entre a interface e os modelos de dados.

3.3 Diagrama de Classes

As principais relações identificadas são:

- Uma Venda contém vários ItemVenda.
- Um ItemVenda referencia um Produto.
- Uma Venda está associada a um Cliente.
- Uma Venda está associada a um Empregado.
- Uma Venda possui um Pagamento.

3.4 Diagrama de Exceções

Foi implementada uma hierarquia de exceções personalizada:

- LojaException
- DadosDuplicadosException
- DadosNaoEncontradosException

- DadosInvalidosException
- StockInsuficienteException

3.5 Diagrama Global

Fluxo principal da aplicação:

Utilizador → View → Controller → Gestores → Modelos

4. Funcionalidades

4.1 Gestão de Clientes

- Adicionar clientes
- Remover clientes
- Pesquisar clientes
- Listar clientes

4.2 Gestão de Empregados

- Adicionar empregados
- Remover empregados
- Pesquisar empregados
- Listar empregados

4.3 Gestão de Produtos

- Adicionar produtos
- Remover produtos
- Atualizar preços
- Atualizar stock
- Consultar produtos

4.4 Gestão de Vendas

- Criar vendas
- Associar clientes
- Associar empregados
- Adicionar produtos
- Calcular total
- Registrar pagamento

5. Diagramas de Sequência

5.1 Criar Venda

1. O utilizador inicia a operação.
2. A View recolhe os dados necessários.
3. O Controller valida os dados.
4. O gestor de clientes identifica o cliente.
5. O gestor de produtos valida os artigos.
6. O gestor de vendas cria a venda.
7. O pagamento é processado.
8. O stock é atualizado.

5.2 Atualizar Stock

1. O utilizador seleciona um produto.
2. O Controller solicita a atualização.
3. O gestor de produtos altera a quantidade disponível.
4. A informação é devolvida à interface.

6. Discussão

A arquitetura MVC adotada permitiu reduzir o acoplamento entre componentes e melhorar a organização geral do código.

A utilização de gestores especializados contribuiu para uma separação clara das responsabilidades, enquanto o sistema de exceções aumentou significativamente a robustez da aplicação.

8. Conclusão

A terceira iteração permitiu consolidar a arquitetura da aplicação e introduzir funcionalidades essenciais ao funcionamento de uma loja de videogames. O sistema desenvolvido evidencia a aplicação prática dos conceitos abordados na unidade curricular, nomeadamente a programação orientada a objetos, a modularização, o tratamento de exceções e a separação de responsabilidades, cumprindo os objetivos estabelecidos na fase inicial do projeto.